Übungsblatt 4: Gemeinsame Daten, Mutex und Prioritätsinversion

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Eine mehrteilige Änderung schützen (8 Punkte)

Ein gemeinsamer Kontostand ist 17. Im folgenden abstrakten Maschinenmodell lädt Einzahler A den Wert und berechnet privat $17 + 4$. A wird unterbrochen. Entnehmer B liest 17, berechnet $17 - 2$ und schreibt zurück. Danach schreibt A seinen gesicherten Wert. Es gibt keine weiteren Zugriffe.

1. Notieren Sie Registerwerte und Speicher nach jedem Schritt. Vergleichen Sie den Endwert mit dem korrekten Gesamtergebnis. (3 Punkte)
2. Genügt ein Schloss nur um die beiden Stores oder `volatile`? Welche Operation muss gemeinsam geschützt werden? (2 Punkte)
3. Zum Stand gehört das Prüffeld $p = 2b + 1$. Ein Schreiber aktualisiert beides unter einem Mutex, der Leser ohne ihn. Geben Sie einen inkonsistenten Snapshot und eine korrekte Leseweise an. (3 Punkte)

Musterlösung: A hält privat 21, Speicher zunächst 17. B hält 15 und schreibt 15. A überschreibt mit 21. Korrekt wäre $17 + 4 - 2 = 19$. Einzelne geschützte Stores verhindern die veralteten Lesegrundlagen nicht. Der vollständige Lese-/Änder-/Schreibzugriff braucht gemeinsamen Ausschluss. In tatsächlichem C kann ein ungeschütztes Data Race undefiniertes Verhalten verursachen; das Modell ist keine zulässige C-Implementierung. Beim Paar kann der Leser etwa neuen Stand 21 und altes Prüffeld 35 sehen statt 43. Leser und Schreiber müssen denselben Mutex verwenden; beide Felder unter dem Schloss privat kopieren, danach freigeben und außerhalb auswerten.

Aufgabe 2: Maskierungsbudget und Token (8 Punkte)

1. Eine Anforderung trifft bei 11 µs verbleibender Interruptmaskierung ein. Eintritt kostet danach 3 µs, Reaktion 4 µs; alles disjunkt. Prüfen Sie $D = 16\,\mu s$ und bestimmen Sie den maximal zulässigen Maskierungsrest. (3 Punkte)
2. Ein Semaphor hat Maximum vier, anfangs aber zwei Token. A und B erwerben je eines, C versucht Acquire mit Timeout null. B gibt frei, dann versucht C erneut. Geben Sie Zähler und Ergebnisse an. Kennt man damit die Adresse einer freien Ressource? (3 Punkte)
3. Ein zunächst leerer binärer Semaphor bekommt drei Releases ohne Acquire. Wie viele gelingen? Warum ersetzt das keinen Mutex mit Besitzerbindung und Prioritätsvererbung? (2 Punkte)

Musterlösung: $R = 11 + 3 + 4 = 18$ µs, Verletzung um 2 µs; höchstens $16 - 3 - 4 = 9$ µs Maskierungsrest. Tokenfolge $2 \to 1 \to 0$, Cs erster Versuch scheitert ohne Blockieren; Release führt auf 1, der neue erfolgreiche Versuch auf 0. Die Tokenzahl enthält keine Pufferidentität. Beim binären Semaphor gelingt nur das erste Release, zwei scheitern an Vollbelegung. Er signalisiert Verfügbarkeit, enthält aber nicht automatisch die Mutex-Besitzerbindung und -Vererbung.

Aufgabe 3: Drei Schlösser, ein Zyklus (8 Punkte)

T1 hält A und wartet auf B, T2 hält B und wartet auf C, T3 hält C und wartet auf A. Es gibt je eine Instanz und unbegrenztes Warten.

1. Zeichnen und beschriften Sie den Ressourcenwartegraphen. Welche Threads können nicht fortfahren? (3 Punkte)
2. Erklären Sie „No preemption“ als Deadlockbedingung und unterscheiden Sie es von Prozessorpräemption. (2 Punkte)
3. Zeigen Sie, wie eine globale Erwerbsordnung A vor B vor C den Zyklus verhindert. (1 Punkt)
4. T2 bekommt beim Erwerb von C einen Timeout. Was muss sein Fehlerpfad mit B und begonnenen Änderungen tun? Warum beweist sofortiges Wiederholen keinen Fortschritt? (2 Punkte)

Musterlösung: Zyklus $A \to T1 \to B \to T2 \to C \to T3 \to A$; Ressource zum Thread heißt Besitz, Thread zur Ressource Warten. Alle drei bleiben ohne Abbruch blockiert, andere Systemteile können weiterlaufen. No preemption betrifft den fehlenden zwangsweisen Ressourcenentzug. Ein präemptiver CPU-Scheduler löst den Zyklus nicht. Nach globaler Ordnung darf T3 nicht bereits C halten und anschließend A verlangen. Nach Timeout C nicht benutzen, B passend freigeben und Teiländerungen konsistent zurücknehmen/abschließen. Symmetrische wiederholte Konflikte können Livelock verursachen; Prioritätsvererbung entfernt den Zyklus nicht.

Aufgabe 4: Prioritätsinversion mit neuen Jobs (8 Punkte)

Ein Kern, feste Ränge Alarm > Auswertung > Archiv; einmalige Jobs, API- und Wechselkosten null. Archiv startet bei 0, hält einen Lock und braucht insgesamt 6 ms CPU-Zeit bis zur Freigabe. Alarm wird bei 2 ms frei, braucht denselben Lock und danach 2 ms eigene Arbeit. Auswertung wird bei 3 ms frei, benötigt 9 ms und keinen Lock. Für Alarm gilt $D = 9\,\text{ms}$.

1. Zeichnen Sie ohne Vererbung bis 17 ms und berechnen Sie Alarm-Warte- und Antwortzeit. (3 Punkte)
2. Wiederholen Sie mit vererbendem Mutex. Markieren Sie die effektive Prioritätsanhebung. (3 Punkte)
3. Prüfen Sie beide Fristen. Welche Verzögerung verschwindet, welche Restarbeit bleibt? (2 Punkte)

Musterlösung: Ohne Vererbung: Archiv 0–3 (Alarms erfolgloser Versuch bei 2 dauert null), Auswertung 3–12, Archiv 12–15, Alarm 15–17. Alarm wartet $15 - 2 = 13$ ms, Antwort $17 - 2 = 15$ ms; absolute Deadline 11 ms, Miss um 6 ms. Mit Vererbung: Archiv 0–6, ab 2 bis 6 auf Alarmrang; Alarm 6–8, Auswertung 8–17. Lockwarten 4 ms, Antwort 6 ms, Deadline eingehalten. Archiv-Restarbeit von 4 ms bei Anforderung bleibt; Auswertung als zusätzliche Verdrängung entfällt. Gesamt-CPU-Arbeit jeweils 17 ms. Kein Ressourcenzyklus, also hier kein Deadlock.

Aufgabe 5: Freigeben heißt nicht fertig

Ein Thread startet einen asynchronen Transfer mit einem Zeiger auf seinen Ausgabepuffer, gibt den Mutex frei und befüllt denselben Puffer sofort neu. Ein anderer Thread nimmt den Mutex korrekt. Ist der Transferinhalt dadurch geschützt? Formulieren Sie eine Besitzregel bis zum tatsächlichen Abschluss.

Musterlösung: Der Mutex schützt nur teilnehmende Softwarezugriffe, nicht die fortlaufenden Hardwarelesezugriffe. Der Puffer darf bis zum bestätigten Transferabschluss weder ungültig werden noch überschrieben werden. Geeignet ist ein eigener Ausgabebesitzer mit eindeutigem Puffertransfer oder ein entsprechend langer Besitzvertrag; bloß längeres C-Objektleben genügt nicht.

Aufgabe 6: Einen konsistenten Ressourcenstand implementieren

Programmieren, etwa 30–40 Minuten. `lab04_account.c/.h` und `account_student.c` laufen auf CM4 mit CMSIS-RTOS2. Zwei vorbereitete Threads ändern einen anfangs 1000 großen Stand je 100-mal: einer um +3, einer um −2. Zum Stand gehört $check = 2 \cdot balance + 1$. Ein dritter Thread prüft Snapshots und bedient **LD1/PB0** (Paar konsistent), **LD3/PB14** (Fehler) und **LD2/PE1** (Status).

1. Implementieren Sie `Lab04_Adjust(delta)`: Mutex erwerben, Stand und Prüffeld gemeinsam aktualisieren, freigeben. Verwenden Sie `lab04_account_mutex`, `lab04_account` und `Lab04_Check(status)` zur Fehlerprüfung. Das Feld `done` bleibt dabei unverändert.
2. Implementieren Sie `Lab04_Snapshot()`: unter demselben Mutex das gesamte `Lab04_Account` in eine lokale Kopie übernehmen, freigeben, Kopie zurückgeben. LED-Zugriffe und Warteaufrufe gehören nicht in diesen kritischen Abschnitt.
3. Berechnen Sie den Endstand nach beiden abgeschlossenen Arbeitsfolgen. Prüfen Sie `lab04_account.done == 2`, den Endstand und beide LED-Aussagen. Der Ausgangsstand der Startdatei liefert noch keine gültigen Snapshots.
4. Begründen Sie, weshalb ein erfolgreicher Schreiber-Mutex ohne passenden Leserschutz nicht genügt. Führen Sie keine absichtlich ungeschützten C-Datenrennen auf dem Board aus.

Vorbereitete Basis: Projektanleitung, STM32CubeMX und VS Code mit STM32 VS Code Extension für **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Board und USB/ST-LINK genügen. Eigene Implementierung und tatsächlichen Prüfstatus festhalten; Debuggerdiagnosen bei angehaltenem zuständigem Kern lesen.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS und gegebenenfalls CMSIS-RTOS2. Papierwerte sind Lehrannahmen.

Musterlösung:

Musterlösung zu Aufgabe 6

Endstand $1000 + 100 \cdot 3 - 100 \cdot 2 = 1100$, Prüffeld 2201. Alle Teilnehmer am Paar benutzen denselben Mutex. `done` wird von der Basis ebenfalls unter diesem Mutex aktualisiert.

```
#include "lab04_account.h"

void Lab04_Adjust(int32_t delta)
{
    Lab04_Check(osMutexAcquire(lab04_account_mutex, osWaitForever));
    lab04_account.balance += delta;
    lab04_account.check = 2 * lab04_account.balance + 1;
    Lab04_Check(osMutexRelease(lab04_account_mutex));
}

Lab04_Account Lab04_Snapshot(void)
{
    Lab04_Check(osMutexAcquire(lab04_account_mutex, osWaitForever));
    const Lab04_Account copy = lab04_account;
    Lab04_Check(osMutexRelease(lab04_account_mutex));
    return copy;
}
```

Die endlichen Änderungen bleiben im `int32_t`-Wertebereich. Die LED prüft funktionale Invarianten und den Endstand dieses Laufs; daraus entsteht kein Zeit- oder allgemeiner Race-Freiheitsnachweis.